Employee Attrition Prediction & HR Analytics System

Phase-Wise Implementation Guide: ETL Pipeline (Phase 1 of the project)

Source: WA_Fn-UseC_-HR-Employee-Attrition.csv | Stack: Python, pandas, PostgreSQL, SQLAlchemy, Docker

1. Overview

This guide breaks the ETL build into twelve ordered phases (0 to 11). Each phase lists its goal, files, implementation steps, a reference snippet where useful, and an exit check that must pass before the next phase starts. The ETL layer produces analytics.employee_features, the handoff table for the later ML pipeline. No model training, splitting, scaling or resampling happens in this phase.

1.1 Verified facts about the dataset

These numbers come from profiling the uploaded CSV and are used as test oracles throughout.

Check	Result
Rows x columns	1,470 x 35 (header matches the expected list exactly)
Missing values	0 in every column
Exact duplicate rows / duplicate EmployeeNumber	0 / 0 (EmployeeNumber runs 1 to 2068, not contiguous)
Attrition	No 1,233 | Yes 237 | rate 16.12%
OverTime	No 1,054 | Yes 416
Department	Research & Development 961 | Sales 446 | Human Resources 63
Constant columns	EmployeeCount = 1, Over18 = Y, StandardHours = 80
PerformanceRating observed	3 (1,244 rows) and 4 (226 rows) only
Ratio edge cases	TotalWorkingYears = 0 in 11 rows; YearsAtCompany = 0 in 44 rows
File encoding	UTF-8 with BOM: read with encoding="utf-8-sig"

1.2 Architecture

```
Raw CSV -> Extract -> staging.employee_raw (JSONB, lineage)
        -> Transform (rename, trim, cast, flags, features)
        -> Validate (hard + soft rules)
              |-- fail    -> etl.pipeline_runs = FAILED, no analytics writes
              |-- rejects -> etl.rejected_records
              |-- results -> etl.data_quality_results
        -> Upsert into analytics.employee_features -> SQL views / ML handoff
```

1.3 Final project structure

```
employee-attrition-system/
  data/raw/  data/processed/
  src/__init__.py
  src/config/{__init__.py, settings.py, schema_contract.py}
```

```
  src/utils/{__init__.py, logging_config.py}
  src/etl/{__init__.py, extract.py, transform.py, validate.py, load.py, pipeline.py}
  sql/{schemas.sql, staging.sql, analytics.sql, analytics_queries.sql}
  tests/{conftest.py, test_extract.py, test_transform.py,
         test_validation.py, test_pipeline.py}
  .env.example  .gitignore  requirements.txt  Makefile
  README.md  docker-compose.yml
```

Phase 0: Project skeleton and environment

Goal: A runnable empty project with a healthy PostgreSQL container.

Files: folder tree above, requirements.txt, .env.example, .gitignore, docker-compose.yml, settings.py, logging_config.py, Makefile.

Implementation steps

- Copy the CSV into data/raw/. Add .env, data/raw/* and data/processed/* to .gitignore.
- Pin versions in requirements.txt: pandas, numpy, SQLAlchemy (2.x), psycopg2-binary, python-dotenv, pytest. Pinning matters because newer pandas reports text columns as "str" instead of "object", so tests must not assert dtype == object.
- settings.py reads POSTGRES_HOST, POSTGRES_PORT, POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD, RAW_DATA_PATH, REJECT_THRESHOLD_PCT and LOG_LEVEL from the environment. The password has no default.
- docker-compose.yml interpolates ${POSTGRES_PASSWORD}, adds a pg_isready healthcheck and a named volume.
- Name the logger module logging_config.py so it never shadows the standard library.
- Makefile targets: up, down, etl, test.

Reference snippet

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports: ["${POSTGRES_PORT}:5432"]
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      retries: 10
volumes:
  pgdata:
```

Exit check: docker compose up -d reports a healthy container, and importing settings succeeds.

Phase 1: Profiling and data contract

Goal: Turn the profile into a machine-readable contract that every later stage uses.

Files: src/config/schema_contract.py.

Implementation steps

- Store the 35 expected source columns in order, with the source dtype for each.
- Store allowed sets: Attrition and OverTime {Yes, No}; BusinessTravel (3 values); Department (3); EducationField (6); Gender (2); JobRole (9); MaritalStatus (3).
- Store ordinal domains: Education 1-5, JobLevel 1-5, StockOptionLevel 0-3, and 1-4 for EnvironmentSatisfaction, JobInvolvement, JobSatisfaction, RelationshipSatisfaction and WorkLifeBalance.
- PerformanceRating: set the hard rule to the domain you confirm from your data source (observed is only 3-4) and add a soft warning if the observed set changes.
- Store hard numeric bounds (for example Age 18 to 65 as a plausible working range, wider than the observed 18-60) and cross-field rules.
- Record a one-line rationale per rule: "observed in file" or "domain knowledge".
- Record known constants and known NULL-ratio cases.

Reference snippet

```
EXPECTED_COLUMNS = ["Age", "Attrition", "BusinessTravel", ...]  # 35, in file order
ALLOWED = {"Attrition": {"Yes", "No"}, "OverTime": {"Yes", "No"}, ...}
ORDINAL = {"Education": (1, 5), "JobLevel": (1, 5),
           "StockOptionLevel": (0, 3), "WorkLifeBalance": (1, 4), ...}
CONSTANTS = {"EmployeeCount": 1, "Over18": "Y", "StandardHours": 80}
```

Exit check: the contract validates the real CSV with zero violations, matching section 1.1.

Phase 2: Database schemas and DDL

Goal: Create every schema, table, index and role idempotently.

Files: sql/schemas.sql, sql/staging.sql, sql/analytics.sql.

Implementation steps

- schemas.sql: CREATE SCHEMA IF NOT EXISTS staging, analytics, etl.
- etl.pipeline_runs with a CHECK on status (RUNNING, SUCCESS, FAILED, PARTIAL), plus source_file_sha256 and failed_stage.
- etl.data_quality_results and etl.rejected_records for rule outcomes and row-level rejects.
- staging.employee_raw with JSONB raw_record and a unique key on (run_id, source_row_number).
- analytics.employee_features keyed on employee_number, with snake_case source columns, text and boolean versions of the two flags, the two ratio features and lineage columns.
- CHECK constraints on hard domains as a last line of defense. Indexes on attrition_flag, department, job_role and overtime_flag.
- Roles etl_writer (all three schemas) and analytics_reader (SELECT on analytics only).
- The pipeline executes these files at start-up so setup works on any PostgreSQL, including AWS RDS later.

Reference snippet

```
CREATE TABLE IF NOT EXISTS etl.pipeline_runs (
  run_id UUID PRIMARY KEY,
  pipeline_name TEXT NOT NULL,
  source_file TEXT NOT NULL,
```

```
  source_file_sha256 CHAR(64),
  started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  completed_at TIMESTAMPTZ,
  rows_extracted INT, rows_transformed INT,
  rows_loaded INT, rows_rejected INT,
  status TEXT NOT NULL CHECK (status IN
        ('RUNNING','SUCCESS','FAILED','PARTIAL')),
  failed_stage TEXT,
  error_message TEXT
);

CREATE TABLE IF NOT EXISTS staging.employee_raw (
  staging_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  run_id UUID NOT NULL REFERENCES etl.pipeline_runs(run_id),
  source_row_number INT NOT NULL,
  employee_number INT,
  source_file TEXT NOT NULL,
  ingested_at TIMESTAMPTZ NOT NULL,
  row_hash CHAR(64) NOT NULL,
  raw_record JSONB NOT NULL,
  UNIQUE (run_id, source_row_number)
);
```

Exit check: running all DDL files twice produces no errors and no changes.

```
CREATE TABLE IF NOT EXISTS analytics.employee_features (
  employee_number INT PRIMARY KEY,
  age INT CHECK (age BETWEEN 18 AND 65),
  attrition TEXT CHECK (attrition IN ('Yes','No')),
  attrition_flag BOOLEAN NOT NULL,
  over_time TEXT CHECK (over_time IN ('Yes','No')),
  overtime_flag BOOLEAN NOT NULL,
  department TEXT, job_role TEXT,
  -- ... remaining source attributes, snake_case, typed from the contract ...
  company_tenure_ratio NUMERIC(8,4),        -- NULL when total_working_years = 0
  current_role_tenure_ratio NUMERIC(8,4),   -- NULL when years_at_company = 0
  staging_id BIGINT REFERENCES staging.employee_raw(staging_id),
  row_hash CHAR(64) NOT NULL,
  source_file TEXT NOT NULL,
  run_id UUID NOT NULL REFERENCES etl.pipeline_runs(run_id),
  processed_at TIMESTAMPTZ NOT NULL
);
CREATE INDEX IF NOT EXISTS ix_ef_attrition  ON analytics.employee_features (attrition_flag);
CREATE INDEX IF NOT EXISTS ix_ef_department ON analytics.employee_features (department);
CREATE INDEX IF NOT EXISTS ix_ef_job_role   ON analytics.employee_features (job_role);
CREATE INDEX IF NOT EXISTS ix_ef_overtime   ON analytics.employee_features (overtime_flag);
```

Phase 3: Extract

Goal: Safe, auditable ingestion of the raw file with the original data untouched.

Files: src/etl/extract.py, tests/test_extract.py.

Implementation steps

- Check that the path exists, is a file, is readable and has size above zero. Raise specific exceptions: SourceFileNotFound, SourceFileEmpty.
- Read with encoding="utf-8-sig" and dtype=str so nothing is coerced before validation.
- Compare the header with EXPECTED_COLUMNS; raise SchemaMismatchError listing missing and unexpected columns.

- Compute SHA-256 of the file, row count, column count, ingestion timestamp (UTC) and source filename.
- Generate run_id with uuid4().
- Return an ExtractResult dataclass: dataframe, run_id, source_file, sha256, rows, columns, ingested_at.

Reference snippet

```
def extract(path: Path) -> ExtractResult:
    if not path.is_file(): raise SourceFileNotFound(path)
    if path.stat().st_size == 0: raise SourceFileEmpty(path)
    df = pd.read_csv(path, dtype=str, encoding="utf-8-sig",
                     keep_default_na=False)
    check_header(df.columns.tolist())
    if df.empty: raise SourceFileEmpty(path)
    return ExtractResult(df, uuid4(), path.name, sha256_of(path),
                         len(df), df.shape[1], datetime.now(timezone.utc))
```

Exit check: extract returns 1,470 rows and 35 columns; a missing, empty and wrong-header file each raise their own clear error.

Phase 4: Staging load

Goal: Preserve every raw record with lineage before any transformation.

Files: src/etl/load.py (staging functions).

Implementation steps

- Insert a row into etl.pipeline_runs with status RUNNING, the file name and the checksum.
- If the same checksum already has a SUCCESS run, log it. Make the behavior configurable: skip, or reprocess.
- Build one JSONB document per source row using the original string values and original column names.
- Compute row_hash as SHA-256 of the canonical JSON (sorted keys). Add source_row_number (1-based file order) and employee_number (parsed when possible, else NULL).
- Bulk insert with executemany or psycopg2 execute_values and commit. Staging is committed before transform so raw data survives later failures.

Exit check: staging holds 1,470 rows for the run, and each traces to its run_id. A second run adds another 1,470 staging rows and does not touch analytics.

Phase 5: Transform

Goal: Clean, typed, standardized data plus derived analytical features.

Files: src/etl/transform.py, tests/test_transform.py.

Implementation steps

- Rename columns to snake_case using one explicit mapping dictionary (35 entries), for example EmployeeNumber to employee_number and YearsWithCurrManager to years_with_curr_manager.
- Trim whitespace on all text columns. This is a no-op on the current file but the rule stays in place.
- Cast types from the contract. Use pd.to_numeric(errors="raise") inside a wrapper that collects offending rows, so failures become rejects instead of silent NaN values.

- Boolean normalization: attrition_flag = (attrition == "Yes"), overtime_flag = (over_time == "Yes"). Anything not Yes/No is invalid and goes to the reject list, never guessed. Keep the original text columns.
- Duplicate detection: count exact duplicate rows and duplicate employee_number values and log both. Policy: exact duplicates may be dropped with a logged count; conflicting duplicates of one employee_number are rejected for review. Today both counts are 0.
- Null analysis: per column, null count, percent and treatment. Report explicitly that all 35 columns have 0 nulls. Never fill with 0 automatically.
- Ratio features, only with a valid denominator, otherwise NULL.
- Compute row_hash on the cleaned record for change detection in the load.
- Do not scale, encode, split or resample here.

Reference snippet

```
df["company_tenure_ratio"] = np.where(
    df.total_working_years > 0,
    df.years_at_company / df.total_working_years.where(df.total_working_years > 0),
    np.nan)
df["current_role_tenure_ratio"] = np.where(
    df.years_at_company > 0,
    df.years_in_current_role / df.years_at_company.where(df.years_at_company > 0),
    np.nan)
```

Exit check: 1,470 rows in and out; attrition_flag true for 237 rows; overtime_flag true for 416; exactly 11 NULL company_tenure_ratio and 44 NULL current_role_tenure_ratio.

Phase 6: Data quality validation

Goal: Explicit, severity-aware rules that stop bad data reaching analytics.

Files: src/etl/validate.py, tests/test_validation.py.

Implementation steps

- Model each rule as a small object: name, severity (HARD or SOFT), check function, failed-row selector.
- Identity rules (HARD): employee_number not null; employee_number unique.
- Domain rules (HARD): attrition and over_time in {Yes, No}; all categorical columns inside their allowed sets; ordinal columns inside their domains; numeric columns inside hard bounds.
- Cross-field rules (HARD): years_at_company <= total_working_years; years_in_current_role, years_with_curr_manager and years_since_last_promotion <= years_at_company.
- Soft rules (WARN): constant columns detected; PerformanceRating value set differs from the contract; attrition rate outside a configured band.
- Write every rule outcome to etl.data_quality_results. Write failing rows to etl.rejected_records with the rule name.
- Decision logic: any HARD failure at dataset level (for example duplicate ids) marks the run FAILED at stage VALIDATE. Row-level rejects below REJECT_THRESHOLD_PCT allow a PARTIAL run; above it, FAILED.

Reference snippet

```
Rule(name="attrition_valid", severity=HARD,
     check=lambda d: d.attrition.isin(["Yes", "No"]))
Rule(name="tenure_le_total_experience", severity=HARD,
```

```
    check=lambda d: d.years_at_company <= d.total_working_years)
```

Exit check: the real file produces 0 hard failures. Corrupted fixtures (null id, "Maybe" in Attrition, age 150, duplicate id, tenure above total experience) each trigger exactly the intended rule.

Phase 7: Analytics load and pipeline orchestration

Goal: Idempotent load into analytics.employee_features, wired into a single runnable pipeline.

Files: src/etl/load.py (analytics functions), src/etl/pipeline.py.

Implementation steps

- Upsert in one transaction using INSERT ... ON CONFLICT (employee_number) DO UPDATE, guarded by a row_hash comparison so unchanged rows are not rewritten.
- Report inserted, updated and unchanged counts separately. rows_loaded = inserted + updated.
- Pipeline order: start run, extract, stage, transform, validate, load, report, finalize run.
- Wrap each stage in try/except. On error set status FAILED, failed_stage, error_message and completed_at; log the traceback; exit with a non-zero code. Never continue after a hard failure.
- Finalize with SUCCESS, or PARTIAL when rejects are within the threshold.
- Add a CLI: python -m src.etl.pipeline --file data/raw/WA_Fn-UseC_-HR-Employee-Attrition.csv.

Reference snippet

```
INSERT INTO analytics.employee_features (employee_number, ..., row_hash, run_id, processed_at)
VALUES (%(employee_number)s, ..., %(row_hash)s, %(run_id)s, now())
ON CONFLICT (employee_number) DO UPDATE SET
  age = EXCLUDED.age,  -- ... every non-key column ...
  row_hash = EXCLUDED.row_hash,
  run_id = EXCLUDED.run_id,
  processed_at = EXCLUDED.processed_at
WHERE analytics.employee_features.row_hash IS DISTINCT FROM EXCLUDED.row_hash;
```

Exit check: first run loads 1,470 rows; a second run on the same file leaves 1,470 rows with 0 updates; a forced failure in each stage records FAILED with the right failed_stage and leaves analytics unchanged.

Phase 8: Data quality report

Goal: A human-readable and machine-readable summary after each successful run.

Files: report function in pipeline.py (or src/etl/report.py).

Implementation steps

- Include: source, transformed, loaded and rejected row counts; duplicate counts; null count and percent by column; invalid records; attrition distribution and rate; department and job role distributions.
- Attrition rate = attrition Yes / total employees x 100.
- Write data/processed/quality_report_<run_id>.json and .md, plus employee_features_<run_id>.csv as a reproducible snapshot.
- Print the summary to the log at the end of the run.

Reference snippet

```
Total Employees: 1470
Attrition:  Yes: 237   No: 1233   Rate: 16.12%
Department: Research & Development 961 | Sales 446 | Human Resources 63
Nulls: 0 in all 35 columns | Duplicates: 0 | Rejected: 0
```

Exit check: every figure in the report matches section 1.1 exactly.

Phase 9: SQL analytics views

Goal: The 15 dashboard analyses as stable database views.

Files: sql/analytics_queries.sql.

Implementation steps

- Create views in the analytics schema, for example analytics.v_attrition_by_department.
- Cover: total employees; attrition count; attrition rate; department headcount; department attrition; job role attrition; OverTime; JobSatisfaction; WorkLifeBalance; MonthlyIncome (binned); YearsAtCompany (binned); BusinessTravel; Gender; JobLevel; EnvironmentSatisfaction.
- Each grouped view returns headcount, attrition count and attrition rate percent. Guard the division with NULLIF.
- Define income and tenure bins once (for example income in 5,000 steps, tenure in 0-2, 3-5, 6-10, 11+ years).
- Grant SELECT to analytics_reader.

Reference snippet

```
CREATE OR REPLACE VIEW analytics.v_attrition_by_department AS
SELECT department,
       COUNT(*) AS employees,
       COUNT(*) FILTER (WHERE attrition_flag) AS attrition_count,
       ROUND(100.0 * COUNT(*) FILTER (WHERE attrition_flag)
             / NULLIF(COUNT(*), 0), 2) AS attrition_rate_pct
FROM analytics.employee_features
GROUP BY department
ORDER BY attrition_rate_pct DESC;
```

Exit check: the overall rate view returns 16.12, and the counts in every grouped view sum to 1,470.

Phase 10: Automated tests

Goal: Cover every behavior requested, in two tiers.

Files: tests/conftest.py, test_extract.py, test_transform.py, test_validation.py, test_pipeline.py.

Implementation steps

- Unit tier (no database): file validation, schema validation, column normalization, boolean transformation, numeric conversion, duplicate detection, null detection, range validation, feature generation including zero-denominator cases.
- Integration tier (separate test database, for example POSTGRES_DB with a _test suffix): idempotent loading, ETL failure handling per stage, end-to-end run on the real CSV compared with the expected report values.

- Use small in-memory DataFrames in conftest.py as fixtures. Avoid asserting dtype == object.
- Add a CI-friendly target: make test.

Reference snippet

```
def test_second_run_is_noop(db, sample_csv):
    run_pipeline(sample_csv)
    first = count_rows(db, "analytics.employee_features")
    run_pipeline(sample_csv)
    assert count_rows(db, "analytics.employee_features") == first
    assert last_run(db).rows_loaded == 0
```

Exit check: pytest passes with no production credentials required.

Phase 11: README, handoff and release

Goal: A project another engineer, or the ML phase, can pick up cold.

Files: README.md.

Implementation steps

- Write the 14 sections from your spec: overview, business problem, architecture, dataset, data dictionary, extraction, transformation, data-quality rules, PostgreSQL architecture, how to run, example SQL, ETL output, ML handoff, future MLOps.
- Include the Mermaid architecture diagram and flag the three constant columns in the data dictionary.
- ML handoff contract: table analytics.employee_features, key employee_number, target attrition_flag, identifier columns to exclude, zero-variance columns to drop, 16.12% positive class so use stratified splits, resampling only on the training split in the ML phase.
- Document run commands: cp .env.example .env, make up, make etl, make test.

Exit check: a fresh clone runs end to end using only the README.

Summary of phases

Phase	Deliverable	Exit evidence
0	Skeleton, Docker, settings	Healthy Postgres; settings import
1	Data contract	Zero violations on real CSV
2	DDL, indexes, roles	DDL re-runnable
3	Extract	1,470 x 35; clean errors on bad files
4	Staging load	1,470 traceable raw rows
5	Transform	237 / 416 flags; 11 and 44 NULL ratios
6	Validation	0 hard failures; corrupt fixtures caught
7	Analytics load + pipeline	Second run changes nothing
8	Quality report	Matches dataset facts
9	SQL views	Counts reconcile to 1,470
10	Tests	pytest green

Phase	Deliverable	Exit evidence
11	README and handoff	Clean-clone run works

Suggested order of work: complete phases 0 and 1 together, then proceed strictly in sequence. Tell me which phase to build next and I will write its code.